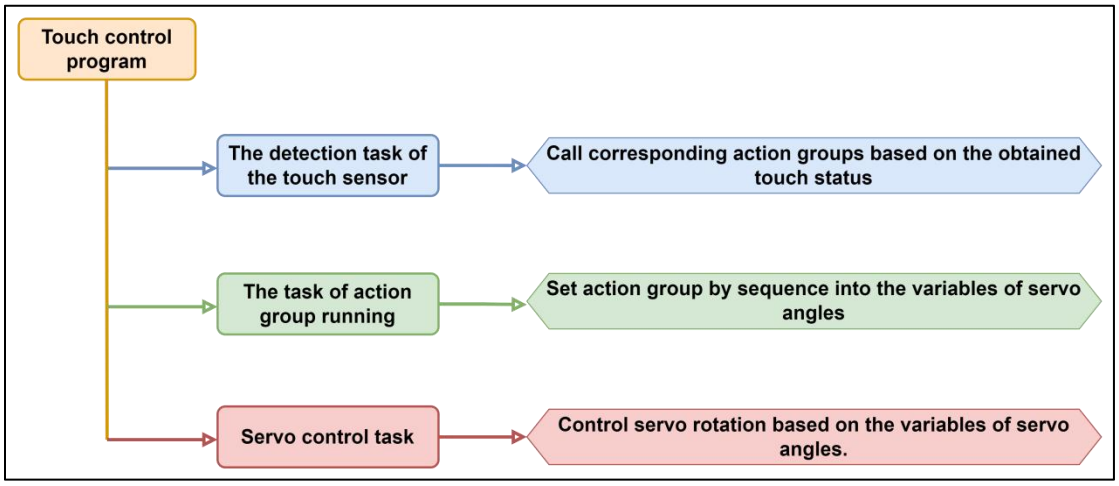
Touch Control

In this section, you will learn how to use the touch sensor to test touch state, realize the action of touching the robotic arm to grasp the square, and touching the robotic arm to place the square. Meanwhile, control the RGB light to change color.

1. Program Flowchart

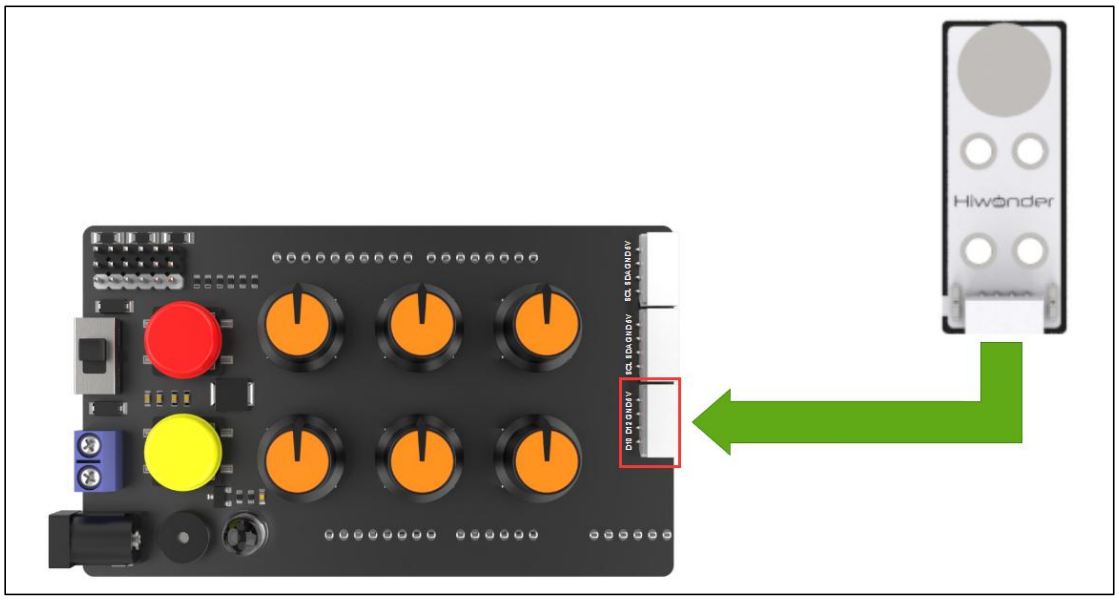


2. Touch Sensor

This is a touch sensor based on the principle of capacitance sensing, which mainly detects human touch or metal through gold-plated contact surface on the sensor.

2.3 Wiring

Connect the ultrasonic module to D10 and D12 interface through 4Pin cable as pictured:



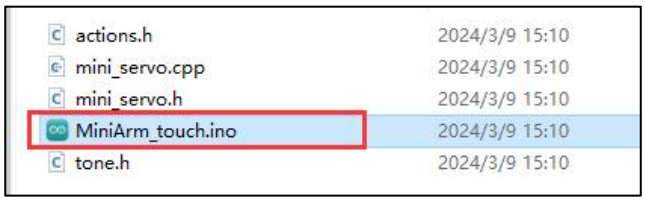
3. Program Download

Note:

◆ Remove the Bluetooth module before downloading the program, otherwise the serial interface conflict will lead to program download failure.

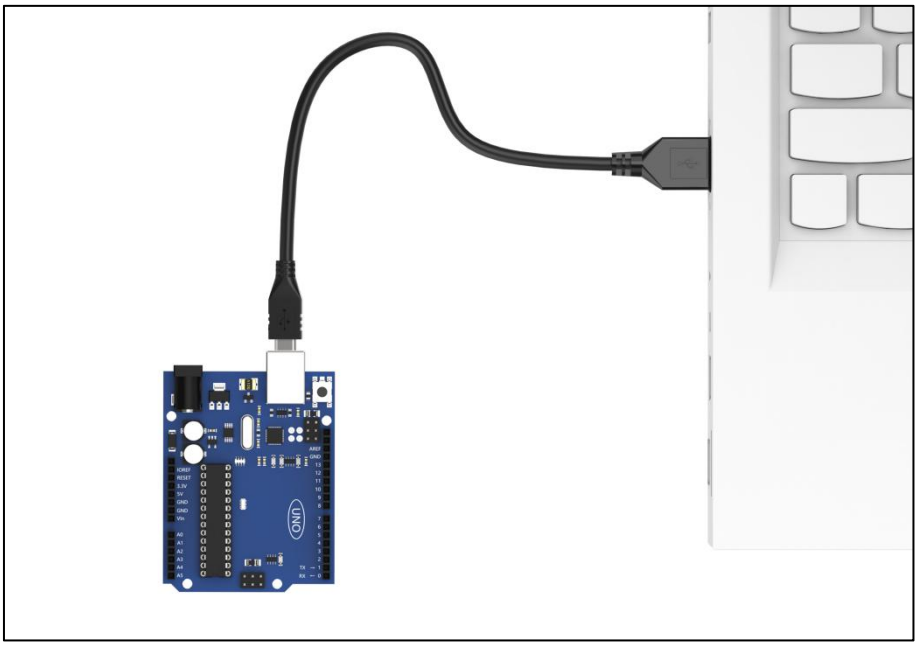
◆ Please turn the switch of the battery box to “OFF” when connect Type-B download cable to avoid the cable touch the power pins of expansion board by mistake, which may cause a short circuit.

1)Locate “MiniArm_Touch\MiniArm_touch.ino” program file in the same path as this document.



actions.h	2024/3/9 15:10
mini_servo.cpp	2024/3/9 15:10
mini_servo.h	2024/3/9 15:10
MiniArm_touch.ino	2024/3/9 15:10
tone.h	2024/3/9 15:10

2)Connect Arduino to the computer through UNO Type-B cable.



3)Click “Select Development Board” option. The software will automatically test current Arduino serial port, then click to connect.

4)Click to download the program to Arduino, then wait for download to complete.

4. Program Outcome

After powering on, the robotic arm will return to the initial position and then proceed to preparatory action for grasping the block.

When you touch the metal surface of the sensor once, the buzzer on the

expansion board will sound and the RGB light will turn red. After that, the robotic arm will execute the action group for grasping the block.

Once the arm has stopped running, touch the metal side of the sensor again, at which point the buzzer will sound again and place the cube on the left side of the arm. At this point, the RGB light on the expansion board will emits red.

5. Program Brief Analysis

5.1 Import Library File

```
#include <FastLED.h> //import LED library
#include <Servo.h> //import servo library
#include "tone.h" //tone library
#include "mini_servo.h"
```

Import the library files required for this program, including the RGB control library and servo control library.

5.2 Define Pins and Create Objects

1)Firstly, define the tones of the two buzzers and the Arduino pins connected to the hardware, mainly consist of six servo pins, one touch sensor pin, one buzzer pin and one RGB light pin.

```
const static uint16_t DOC5[] = { TONE_C5 };
const static uint16_t DOC6[] = { TONE_C6 };

/* pin definition */
const static uint8_t servoPins[5] = { 7, 6, 5, 4, 3};//servo pin definition
const static uint8_t touch = 12;
const static uint8_t buzzerPin = 11;
const static uint8_t rgbPin = 13;
```

2)Create the objects of the RGB light, action group control object and servo control object. Meanwhile, variables controlled by the servo are created. For example, “limit_angle” set angle limits for each joint of robotic arm, and the “servo_angles” array is used for storing actual angle of servo with range from 0 to 180.

```
//RGB light control object
static CRGB rgbs[1];
//action group control object
HW_ACTION_CTL action_ctl;
//servo control object
Servo servos[5];

/* limit for each joint angle */
const uint8_t limt_angles[5][2] = {{0,82},{0,180},{0,180},{0,180},{0,180}};
/* the angle value actually controlled by serv */
static float servo_angles[5] = { 1,40,15,175,65,90 };
```

3)In the “mini_servo.h” action group control object program, the constructor also define the “extended_func_angles” variable. That is the desired angle value of servo. Meanwhile, it declares relevant function used for controlling execute action group and reading deviation.

```
class HW_ACTION_CTL{
  public:
  /* the using angles of secondary development example */
    uint8_t extended_func_angles[5] = { 41 ,12 ,174 ,68 ,84 };
    //Control execution action group
    void action_set(int num);
    int action_state_get(void);
    void action_task(void);
    void read_offset();
    int8_t* get_offset(void);

  private:
    //Action group control variables
    int action_num = 0;
    int8_t servo_offset[5];
    uint8_t eeprom_read_buf[16];
};
```

For the desired value and actual value, you can see the “servo_control” function called in the loop main function.

If you want the servo to approach to the target position gradually, call this line of code “servo_angles[i] = servo_angles[i] * 0.85 + extended_func_angles[i] * 0.15” in the control task function.

This causes the servo to move each time by 85% of the actual value and 15% of the desired value, so that the actual value gets progressively closer to the desired value. When they are equal, the robotic arm stop moving.

```
for (int i = 0; i < 5; ++i) {
  if(servo_angles[i] > action_ctl.extended_func_angles[i])
  {
    servo_angles[i] = servo_angles[i] * 0.9 + action_ctl.extended_func_angles[i] * 0.1;
    if(servo_angles[i] < action_ctl.extended_func_angles[i])
      servo_angles[i] = action_ctl.extended_func_angles[i];
  }else if(servo_angles[i] < action_ctl.extended_func_angles[i])
  {
    servo_angles[i] = servo_angles[i] * 0.9 + (action_ctl.extended_func_angles[i] * 0.1 + 1);
    if(servo_angles[i] > action_ctl.extended_func_angles[i])
      servo_angles[i] = action_ctl.extended_func_angles[i];
  }
  servo_angles[i] = servo_angles[i] < limt_angles[i][0] ? limt_angles[i][0] : servo_angles[i];
  servo_angles[i] = servo_angles[i] > limt_angles[i][1] ? limt_angles[i][1] : servo_angles[i];
  servos[i].write(i == 0 || i == 5 ? 180 - servo_angles[i] : servo_angles[i]);
}
```

4)The following section declares task function used for executing different control task. The “servo_control” function is used for controlling servo, the “play_tune()” function is used for setting buzzer sound, the “tune_task()” function is used for controlling buzzer sound, the “touch_task()” is used for testing the touch sensor state and call execution action group.

```
static void servo_control(void); /* servo control */
void play_tune(uint16_t *p, uint32_t beat, uint16_t len); /* buzzer control interface */
void tune_task(void); /* buzzer control task */
void touch_task(void); /* touch sensor detection task */
```

5.3 Initialization Settings

1)Initialize related hardware equipment in “setup()” function. The first is serial interface, which sets the baud rate of communication to 115200 and the timeout for reading data to 500ms.

```
void setup() {
  // put your setup code here, to run once:
  Serial.begin(115200);
  // set the elapsed time for reading data from the serial port
  Serial.setTimeout(500);
```

2)Assign servo IO port. Use the FastLED library to initialize RGB lights on expansion board, and connect it to RGB pin. Then set the color to green by “rgbs[0] = CRGB(0, 255, 0)” and use “FastLED.show” function to display the set color.

```
// assign servo IO port
for (int i = 0; i < 5; ++i) {
  servos[i].attach(servoPins[i],500,2500);
}

//initialize and control RGB light
FastLED.addLeds<WS2812, rgbPin, GRB>(rgbs, 1);
rgbs[0] = CRGB(0, 0, 100);
FastLED.show();
```

3)Initialize the touch sensor and read deviation data of servo

```
//initialize touch sensor
pinMode(touch, INPUT);
//read deviation data of servo
action_ctl.read_offset();
```

4)Set the pin of buzzer to output mode and call the “tong()” function to activate the buzzer. Then call the “noTone()” function to stop activating buzzer afteer a 100ms delay.

```
//initialize buzzer and beep one time
pinMode(buzzerPin, OUTPUT);
tone(buzzerPin, 1000);
delay(100);
noTone(buzzerPin);
```

5.4 Touch Sensor Detection Task

After initialization, call the ultrasonic task function “touch_task” in the loop main function, which is used for testing the state of touch sensor and calling action group by the state.

◆ The function defines the variable “last_tick” firstly, which is mainly used for calculating the time interval.

◆ Next, check if 100ms have passed since the last call to the “ultrasound_task” function. If it have passed, the following code is executed. Otherwise, the function returns directly. The “millis” function is the time since the Arduino board started. In this way, the function ensures that the task of

detecting the distance is performed every 100ms.

◆ Uses the “digitalRead” function to define the touch sensor as 0 when the metal face is pressed and 1 when the metal face is not pressed.

```
// touch sensor detection task
void touch_task(void)
{
  static uint32_t last_tick = 0;
  static uint8_t step = 0;
  static uint8_t act_num = 0;
  static uint32_t delay_count = 0;
  static uint8_t turn;
  //current stage. 0 represents grasping stage; 1 represents placing stage
  static uint8_t stage = 0;


  // time interval is 100ms
  if (millis() - last_tick < 100) {
    return;
  }
  last_tick = millis();
```

5.5 Execution Feedback

1)Following the steps, firstly read the level of the touch sensor pin. If the value of the button is equal to 0, that is to say the button is pressed, the RGB lights up red. After the buzzer emits a beep, determine whether to enter the prevention stage or the grasping stage based on the “stage” variable. If entering the placing stage, set to call action group 2; if entering the grasping stage, set to call action group 1.

```
turn = digitalRead(touch);

switch(step)
{
  case 0:
    if(turn == 0) //if a button press is detected
    {
      rgbs[0].r = 250;
      rgbs[0].g = 0;
      rgbs[0].b = 0;
      FastLED.show();
      play_tune(DOC6, 300u, 1u);
      if(stage)//placing stage
      {
        act_num = 2;
        stage = 0;
      }
      else    //grasping stage
      {
        act_num = 1;
        stage = 1;
      }
      count = 0;
      step++;
    }else //if touch is not detected
    {
      rgbs[0].r = 0;
      rgbs[0].g = 0;
      rgbs[0].b = 0;
      FastLED.show();
    }
    break;
```

2)Enter case 1 and perform delay wait based on the value of “delay_count”. Since each time enter the case 1, the tilt angle detection task is called. And 100ms delay is performed each time when you enter the button detection task. Therefore, the case 1 will delay for 1000ms firstly. Then the action group running flag is set by the “action_set()” function. After that, it will jump to the next stage.

```
case 1: //wait for 1s and place the wood block to the specified position
  delay_count++;
  if(delay_count > 10)
  {
    delay_count = 0;
    // run action group
    action_ctl.action_set(act_num);
    act_num = 0;
    step++;
  }
  break;
```

In the action group running task under the “mini_servo.cpp” file, the value of the “action_num” will be determined directly. If the value is not 0, the action group will run.

```
class HW_ACTION_CTL{
  public:
  /* the angle used in secondary development routine */
    uint8_t extended_func_angles[5] = { 1,40,15,175,65,90 };
    //control and perform action group
    void action_set(int num);
    int action_state_get(void);
    void action_task(void);
    void read_offset(void);
    int8_t* get_offset(void);

  private:
    //action group control variables
    int action_num = 0;
    int8_t servo_offset[5];
    uint8_t eeprom_read_buf[16];
};
```

3)Case 2 is the stage where it waits for the action to finish running. After completing the recognition, waiting, and calling the action group as mentioned above, it checks if the “action_num” variable is 0 to determine if the action group has finished running. After the action group finishes, the step stage variable is reset to 0.

```
case 2: //wait for resetting the action state
  if(action_ctl.action_state_get() == 0)
  {
    step = 0;
  }
  break;
default:
  step = 0;
  break;
```

5.6 Servo Control Task

1)In the servo control task, the same variable “last_tick” is used for delay processing.

```
// servo control task
void servo_control(void) {
  static uint32_t last_tick = 0;
  if (millis() - last_tick < 20) {
    return;
  }
  last_tick = millis();
```

2）Next, get the desired value of the servo through “action_ctl” object. Note that the desired value at this point is the result of the desired value plus the deviation outcome of servo. So this desired value is the actual desired value. Finally, the actual desired value of the servo is assigned to the actual control angle. Then pass it to the servo object to actually control the rotation of the servo.

```
  for (int i = 0; i < 5; ++i) {
    if(servo_angles[i] > action_ctl.extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.9 + action_ctl.extended_func_angles[i] * 0.1;
      if(servo_angles[i] < action_ctl.extended_func_angles[i])
        servo_angles[i] = action_ctl.extended_func_angles[i];
    }else if(servo_angles[i] < action_ctl.extended_func_angles[i])
    {
      servo_angles[i] = servo_angles[i] * 0.9 + (action_ctl.extended_func_angles[i] * 0.1 + 1);
      if(servo_angles[i] > action_ctl.extended_func_angles[i])
        servo_angles[i] = action_ctl.extended_func_angles[i];
    }
    servo_angles[i] = servo_angles[i] < limt_angles[i][0] ? limt_angles[i][0] : servo_angles[i];
    servo_angles[i] = servo_angles[i] > limt_angles[i][1] ? limt_angles[i][1] : servo_angles[i];
    servos[i].write(i == 0 || i == 5 ? 180 - servo_angles[i] : servo_angles[i]);
  }
}
```

Let’s further explain the ideal desired value and actual desired value. In the “action_ctl” object, “extended_func_angles” is the ideal desired value, which excludes deviation due to servo structure. In the actual project,

servos often have a certain range of deviations due to their mechanical structure. To achieve more accurate control of the servo angle, we calculate the additional angle needed for the servo to reach the ideal position by adjusting for this deviation. This additional angle can be positive, negative, or zero. Then, during actual servo control, add this deviation value to the ideal expected angle, resulting in the actual expected angle.

6. Function Extension

The touching state of the touch sensor can be modified to a long press. The specific modifications can be referred in the following steps:

1)Locate the section of the program that detects touch status. You can see that the sensor status is obtained firstly and assigned to the “turn” variable. Then, the touch sensor is detected by checking the value of “turn”.

```
turn = digitalRead(touch);

switch(step)
{
  case 0:
    if(turn == 0) //if a button press is detected
    {
      rgbs[0].r = 250;
      rgbs[0].g = 0;
      rgbs[0].b = 0;
      FastLED.show();
      play_tune(DOC6, 300u, 1u);
```

2)Here, we create a variable “count” as the fixed interval for detection. Firstly, at line 105, we detect the touch status for the first time, and then we check the value of the count variable. Each time the count is less than 8, it increments by 1. As we know from the previous content, the system delays by 100ms each time it reaches the button detection part. Therefore, when “count” equals to 8, it equivalent to 800ms delay. That is to say, after we press on the touch piece for about 800ms, the block will be picked up and placed in the action group on the left.

```
static uint8_t count = 0;
turn = digitalRead(touch);

switch(step)
{
  case 0:
    if(turn == 0) //if a button press is detected
    {
      if(count < 8){
        count++;
      }else{
        rgbs[0].r = 250;
        rgbs[0].g = 0;
        rgbs[0].b = 0;
        FastLED.show();
        play_tune(DOC6, 300u, 1u);
        if(stage)//placing stage
        {
          act_num = 2;
          stage = 0;
        }
        else    //grasping stage
        {
          act_num = 1;
          stage = 1;
        }
        count = 0;
        step++;
        }
```

7. FAQ

Q: since the code was upload, touch sensor does not respond.

A: please check that you have connected the 4-pin cable to the correct IO interface.

Q: the touch sensor is sometimes accurate when touched, and sometimes inaccurate.

A: Please ensure that the sensor contact surface or your fingers are not contaminated.